

gurps-ink

Typeset *GURPS* game aids
Version 0.1.0

Contents

1 Introduction	1
2 Quick start	1
3 Building characters	2
4 Reference	2
5 Legal	15

1 Introduction

gurps-ink helps you write home-made *GURPS* material: adventures, NPC handouts, house-rule documents. It gives you

- dice in *GURPS* notation (3d-1) and book references (*GURPS Basic Set* p. 16);
- the boilerplate the Steve Jackson Games online policy asks for;
- NPC stat blocks that work out point costs, secondary characteristics and basic damage for you.

It is a port of the LaTeX package [gurps](#).

2 Quick start

```
#import "@preview/gurps-ink:0.1.0": *

#let napoleon = character(
  name: "Napoleon",
  st: 9, hp: 12,
  advantage("Natural afro", 1),
  quirk("Big teeth"),
  skill("Nunchuck", 16, "DX/E"),
  melee-attack("Punch", 18, [#dice(1, -2) cr], reach: "C"),
)

#stat-block(napoleon)

#sjgames-disclaimer
```

Napoleon (16 points)
ST 9 [-10]; DX 10 [0]; IQ 10 [0]; HT 10 [0].
HP 12 [6]; Will 10 [0]; Per 10 [0]; FP 10 [0].
Basic Speed 5.00 [0]; Basic Move 5 [0]; Dodge 8.
SM 0; DR 0; thr 1d-2; sw 1d-1.
Advantages: Natural afro [1].
Quirks: Big teeth [-1].
Skills: Nunchuck-16 [20].
Attacks:
 Punch (18): 1d-2 cr. Reach C.

3 Building characters

A character is built in two steps. `character()` collects everything and works out the numbers; it returns a plain dictionary. `stat-block()` turns that dictionary into content. Keeping them apart means you can define a character once and print it, or quote its numbers, anywhere:

```
#let guard = character(st: 12, skill("Broadsword", 13, "DX/A"))
The guard swings for #dice(..guard.sw) and has
#level-of(guard, "HP") HP.
```

Only say what differs from an average human. Leave a cost out and it is calculated; give one and it is used as-is. Traits whose cost cannot be calculated (advantages without points) show [?] and count as 0.

3.1 Coming from the LaTeX package

LaTeX	Typst
<code>\gurps</code>	<code>#gurps</code>
<code>\dice{3}[-1]</code>	<code>#dice(3, -1)</code>
<code>\gurpsbook{Zombies}[3]</code>	<code>#gurps-book("Zombies", 3)</code>
<code>\SJGamesOnlinePolicyDisclaimer</code>	<code>#sjgames-disclaimer</code>
<code>\SJGamesOnlinePolicyNotice</code>	<code>#sjgames-notice</code>
<code>\SJGamesOnlinePolicyGameAid{Me}</code>	<code>#sjgames-game-aid[Me]</code>
<code>\begin{character} ... \end{character}</code>	<code>#stat-block(character(...))</code>
<code>\begin{character*}[key]</code>	<code>#let key = character(...)</code>
<code>\GCPrintCharacter[key]</code>	<code>#stat-block(key)</code>
<code>\ST{13}[25]</code>	<code>st: (level: 13, points: 25)</code>
<code>\skill{Stealth}[DX/Average]{12}</code>	<code>skill("Stealth", 12, "DX/A")</code>
<code>\levelledadvantage{Magery}{2}[25]</code>	<code>advantage("Magery", 25, level: 2)</code>
<code>\meleeattack{name=..., level=...}</code>	<code>melee-attack(name, level, damage, reach: ...)</code>
<code>\GCGet{ST}\GCRresult</code>	<code>#level-of(key, "ST")</code>
<code>\GCTotalPoints</code>	<code>#total-points(key)</code>
<code>\GCAddToLevel{HP}{4}</code>	<code>pass the final value, e.g. hp: 16</code>

Reading characters from GCS files is not supported yet.

4 Reference

4.1 Text and dice

4.1.1 dice

Dice in GURPS notation: the 6 of d6 is dropped and a modifier is written straight after the d. The result never breaks across lines.

```
#dice(3) \
#dice(2, 1) \
#dice(4, -1) \
#dice(1, 0)
```

```
3d
2d+1
4d-1
1d
```

The modifier is optional. Zero prints nothing, negative numbers use a true minus sign. Content modifiers (e.g. \$m\$) get a leading +.

Combine with `thrust()` or `swing()` by spreading: `#dice(..thrust(13))`.

4.1.1.1 Parameters

```
dice(
  count: int content,
  ..modifier: int content
) -> content
```

4.1.1.1.1 count int or content

Number of dice. Must be a non-negative integer, or content for symbolic notation such as \$n\$.

4.1.1.1.2 ..modifier int or content

Optional modifier added to the roll. Defaults to 0.

4.1.2 gurps-book

A reference to a GURPS book, in SJ Games house style: the title in bold italics, optionally followed by a page reference.

```
#gurps-book("High Tech") \
#gurps-book("Zombies", 3) \
#gurps-book("Warehouse 23", "1, 3-5") \
#gurps-book("Basic Set", (16, 170))
```

GURPS High Tech
GURPS Zombies p. 3
GURPS Warehouse 23 pp. 1, 3-5
GURPS Basic Set pp. 16, 170

A single page (an int, or a string of digits) gets “p.”; anything else gets “pp.”. An array of pages is joined with commas.

4.1.2.1 Parameters

```
gurps-book(
  title: str content,
  ..pages: int str content array
) -> content
```

4.1.2.1.1 title `str` or `content`

Book title without the leading “GURPS”.

4.1.2.1.2 ..pages `int` or `str` or `content` or array

Optional page or pages.

4.1.3 sjgames-game-aid

The notice for a free game aid made without an official license.

```
#sjgames-game-aid[Jane Doe]
```

GURPS is a trademark of [Steve Jackson Games](#), and its rules and art are copyrighted by [Steve Jackson Games](#). All rights are reserved by Steve Jackson Games. This game aid is the original creation of Jane Doe and is released for free distribution, and not for resale, under the permissions granted in the [Steve Jackson Games Online Policy](#).

4.1.3.1 Parameters

```
sjgames-game-aid(author: str content) -> content
```

4.1.3.1.1 author `str` or `content`

Who made the game aid.

4.1.4 gurps `content`

The name ***GURPS*** in bold italics, as the SJ Games online policy requires for trademarks. Use it like any other piece of content.

```
#gurps is a generic, universal roleplaying system.
```

GURPS is a generic, universal roleplaying system.

To link it to the GURPS home page, wrap it in link: `#link("http://www.sjgames.com/gurps/", gurps)`.

4.1.5 sjgames content

The name “Steve Jackson Games”.

```
#gurps is published by #sjgames.
```

GURPS is published by Steve Jackson Games.

4.1.6 sjgames-disclaimer content

The disclaimer the SJ Games online policy asks fan material to carry.

```
#sjgames-disclaimer
```

The material presented here is my original creation, intended for use with the **GURPS** system from [Steve Jackson Games](#). This material is not official and is not endorsed by Steve Jackson Games.

4.1.7 sjgames-notice content

The trademark notice the SJ Games online policy asks fan material to carry. “The art here” is omitted because no SJ Games art is included.

```
#sjgames-notice
```

GURPS is a registered trademark of [Steve Jackson Games](#), and is copyrighted by Steve Jackson Games. All rights are reserved by Steve Jackson Games. This material is used here in accordance with the SJ Games [online policy](#).

4.2 Damage

4.2.1 thrust

Thrust damage for a given ST, as a (count, modifier) pair that can be spread into `dice()`.

```
#thrust(10) \  
#dice(..thrust(10))
```

```
(1, -2)
1d-2
```

Covers ST 1–100 using the Basic Set table. Between the table's rows above ST 40 (45, 50, ...) the lower row applies. Panics outside that range: set `thr`: explicitly on the character instead.

4.2.1.1 Parameters

```
thrust(st: int) -> array
```

4.2.1.1.1 st int

Striking strength.

4.2.2 swing

Swing damage for a given ST, as a (count, modifier) pair that can be spread into `dice()`. Same range rules as `thrust()`.

```
#dice(..swing(10))
```

```
1d
```

4.2.2.1 Parameters

```
swing(st: int) -> array
```

4.2.2.1.1 st int

Striking strength.

4.3 Characters

4.3.1 advantage

An advantage.

```
#let afro = advantage("Natural afro", 1)
#let magery = advantage("Magery", 25, level: 2)
#afro.points, #magery.level
```

```
1, 2
```

4.3.1.1 Parameters

```
advantage(
  name: str content,
  ..points: int,
  level: int none
) -> dictionary
```

4.3.1.1.1 **..points** `int`

Optional point cost. Shown as [?] and counted as 0 when omitted.

4.3.1.1.2 **level** `int` or `none`

Level for levelled advantages, e.g. Magery 2.

Default: `none`

4.3.2 **disadvantage**

A disadvantage. The point cost, if given, must be zero or negative.

```
#disadvantage("Bad Temper", -10).points
```

-10

4.3.2.1 **Parameters**

```
disadvantage(  
    name: str content,  
    ..points: int,  
    level: int none  
) -> dictionary
```

4.3.2.1.1 **..points** `int`

Optional point cost (≤ 0).

4.3.2.1.2 **level** `int` or `none`

Level for levelled disadvantages.

Default: `none`

4.3.3 **perk**

A perk. Always costs 1 point.

4.3.3.1 **Parameters**

```
perk(name: str content) -> dictionary
```

4.3.4 quirk

A quirk. Always costs -1 point.

4.3.4.1 Parameters

```
quirk(name: str content) -> dictionary
```

4.3.5 skill

A skill at a given level.

The third argument says what the skill costs. Either give the points directly, or give the controlling attribute and difficulty as GURPS writes them ("DX/E", "IQ/VH", "Per/Average") and the cost is worked out by `character()`. The base may also be another skill or spell on the same character.

Difficulties: E/Easy, A/Average, H/Hard, VH/Very Hard, W/Wildcard.

```
#skill("Stealth", 12, "DX/A") \  
#skill("Brawling", 14, 4) \  
#skill("Carousing", 10)
```

```
(  
  kind: "skill",  
  name: "Stealth",  
  level: 12,  
  points: none,  
  base: "DX",  
  difficulty: "A",  
)  
(  
  kind: "skill",  
  name: "Brawling",  
  level: 14,  
  points: 4,  
  base: none,  
  difficulty: none,  
)  
(  
  kind: "skill",  
  name: "Carousing",  
  level: 10,  
  points: none,  
  base: none,  
  difficulty: none,  
)
```


4.3.5.1 Parameters

```
skill(  
  name: str content,  
  level: int,  
  ..cost: int str  
) -> dictionary
```

4.3.5.1.1 level int

Effective skill level.

4.3.5.1.2 ..cost int or str

Optional cost: points, or "<base>/<difficulty>".

4.3.6 spell

A spell. Works exactly like `skill()`.

```
#spell("Fireball", 14, "IQ/H")
```

```
(  
  kind: "spell",  
  name: "Fireball",  
  level: 14,  
  points: none,  
  base: "IQ",  
  difficulty: "H",  
)
```

4.3.6.1 Parameters

```
spell(  
  name: str content,  
  level: int,  
  ..cost: int str  
) -> dictionary
```

4.3.7 melee-attack

A melee attack.

```
#melee-attack("Punch", 18, [#dice(1, -1) cr], reach: "C")
```

```
(
  kind: "melee-attack",
  name: "Punch",
  level: 18,
  damage: sequence(box(body: sequence([1], [d], [-1])), [ ], [cr]),
  notes: none,
  reach: "C",
)
```

4.3.7.1 Parameters

```
melee-attack(
  name: str content,
  level: int,
  damage: str content,
  reach: str content none,
  notes: str content none
) -> dictionary
```

4.3.7.1.1 level int

Effective skill with the attack.

4.3.7.1.2 damage str or content

Damage, e.g. [#dice(2, 1) cut].

4.3.7.1.3 notes str or content or none

Free-form notes printed after the attack.

Default: none

4.3.8 ranged-attack

A ranged attack.

```
#ranged-attack("Bow", 14, [#dice(1, 2) imp], range: "150/200")
```

```
(
  kind: "ranged-attack",
  name: "Bow",
  level: 14,
  damage: sequence(box(body: sequence([1], [d], [+2])), [ ], [imp]),
  notes: none,
  range: "150/200",
)
```

4.3.8.1 Parameters

```
ranged-attack(  
  name: str content ,  
  level: int ,  
  damage: str content ,  
  range: str content none ,  
  notes: str content none  
) -> dictionary
```

4.3.9 character

Builds a character. Only state what differs from the defaults: an average human (all attributes 10) is `character()`.

Every attribute argument takes a level, or a dictionary (`level: ...`, `points: ...`) to override the calculated cost (useful for discounts such as SM-reduced ST). Defaults and costs follow the Basic Set:

Argument	Default	Cost per level
st, ht	10	10
dx, iq	10	20
hp	ST	2
will, per	IQ	5
fp	HT	3
basic-speed	(DX+HT)/4	20 per 1.00
basic-move	[Basic Speed]	5

Traits, skills, spells and attacks are passed as positional arguments.

```
#let napoleon = character(  
  name: "Napoleon",  
  st: 9, hp: 12,  
  advantage("Natural afro", 1),  
  quirk("Big teeth"),  
  skill("Nunchuck", 16, "DX/E"),  
)  
#napoleon.attributes.HP \  
#total-points(napoleon) points
```

```
(level: 12, points: 6)  
16 points
```

The result is a dictionary with keys `kind` ("character"), `name`, `attributes` (keyed ST, DX, IQ, HT, HP, Will, Per, FP, Basic Speed, Basic Move; each (`level:`, `points:`)), `dodge`, `sm`, `dr`, `thr`, `sw`, and arrays `advantages`, `perks`, `disadvantages`, `quirks`, `skills`, `spells`, `attacks`. Read values with `level-of()`.

4.3.9.1 Parameters

```
character(  
  name: str content none ,  
  st: int dictionary ,  
  dx: int dictionary ,  
  iq: int dictionary ,  
  ht: int dictionary ,  
  hp: auto int dictionary ,  
  will: auto int dictionary ,  
  per: auto int dictionary ,  
  fp: auto int dictionary ,  
  basic-speed: auto int float dictionary ,  
  basic-move: auto int dictionary ,  
  dodge: auto int ,  
  sm: int ,  
  dr: int str content ,  
  thr: auto array content ,  
  sw: auto array content ,  
  ..traits: dictionary  
) -> dictionary
```

4.3.9.1.1 name str or content or none

Shown as the stat block title.

Default: none

4.3.9.1.2 basic-speed auto or int or float or dictionary

In steps of 0.25.

Default: auto

4.3.9.1.3 dodge auto or int

Defaults to [Basic Speed] + 3. Has no point cost.

Default: auto

4.3.9.1.4 sm int

Size Modifier.

Default: 0

4.3.9.1.5 dr int or str or content

Damage Resistance, e.g. 2 or [3 (torso only)].

Default: 0

4.3.9.1.6 thr auto or array or content

Thrust damage: a (count, modifier) pair or content. Defaults to `thrust()` of ST.

Default: `auto`

4.3.9.1.7 sw auto or array or content

Swing damage, like thr. Defaults to `swing()` of ST.

Default: `auto`

4.3.9.1.8 ..traits dictionary

Results of `advantage()`, `disadvantage()`, `perk()`, `quirk()`, `skill()`, `spell()`, `melee-attack()` and `ranged-attack()`, in any order.

4.3.10 level-of

The level of an attribute, skill or spell, looked up by name. Also knows "Dodge" and "SM". Panics, listing what exists, if the name is unknown.

```
#let c = character(dx: 12, skill("Stealth", 13, "DX/A"))
#level-of(c, "Basic Speed"), #level-of(c, "Stealth")
```

5.5, 13

4.3.10.1 Parameters

```
level-of(
  char: dictionary,
  name: str
) -> int float
```

4.3.10.1.1 char dictionary

A result of `character()`.

4.3.10.1.2 name str

e.g. "ST", "Basic Move", "Stealth".

4.3.11 total-points

Total character points: the sum of every known cost. Traits without a cost count as 0.

```
#total-points(character(st: 12, dx: 11))
```

40

4.3.11.1 Parameters

```
total-points(char: dictionary) -> int
```

4.3.11.1.1 char dictionary

A result of `character()`.

4.4 Stat blocks

4.4.1 stat-block

Typesets a character from `character()` as a stat block in the style of SJ Games supplements.

Lines, in order: the title (name and total points), the four basic attributes, HP/Will/Per/FP, Basic Speed/Basic Move/Dodge, SM/DR/thr/sw, then labelled lists of advantages, perks, disadvantages, quirks, skills, spells and attacks. Empty lists are left out. Lists are sorted by name. Skills and spells read Name - level, as in the books.

```
#stat-block(character(  
  name: "Napoleon",  
  st: 9, hp: 12,  
  advantage("Natural afro", 1),  
  quirk("Big teeth"),  
  skill("Nunchuck", 16, "DX/E"),  
  melee-attack("Punch", 18, [#dice(5) cr], reach: "C, 1",  
    notes: [Believe it!]),  
))
```

Napoleon (16 points)

ST 9 [-10]; DX 10 [0]; IQ 10 [0]; HT 10 [0].

HP 12 [6]; Will 10 [0]; Per 10 [0]; FP 10 [0].

Basic Speed 5.00 [0]; Basic Move 5 [0]; Dodge 8.

SM 0; DR 0; thr 1d-2; sw 1d-1.

Advantages: Natural afro [1].

Quirks: Big teeth [-1].

Skills: Nunchuck-16 [20].

Attacks:

Punch (18): 5d cr. Reach C, 1. Believe it!

To frame it, wrap it: `#block(stroke: 0.5pt, inset: 8pt)[#stat-block(c)]`.

4.4.1.1 Parameters

```
stat-block(  
  char: dictionary ,  
  show-points: bool  
) -> content
```

4.4.1.1.1 char dictionary

A result of `character()`.

4.4.1.1.2 show-points bool

Print point costs ([10]) and the total. Turn off for a monster manual feel.

Default: `true`

5 Legal

The material presented here is my original creation, intended for use with the *GURPS* system from [Steve Jackson Games](#). This material is not official and is not endorsed by Steve Jackson Games.

GURPS is a registered trademark of [Steve Jackson Games](#), and is copyrighted by Steve Jackson Games. All rights are reserved by Steve Jackson Games. This material is used here in accordance with the SJ Games [online policy](#).